

Performance Analysis of Multi-Threaded Monte Carlo π Estimation: A Comparative Study of Synchronization Overhead

Long Le Hoang^{1*}, Phuoc Nguyen Huu^{1*}, An Nguyen Gia^{1*}
and Minh Nguyen Hoang^{1*}

^{1*}Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): long.le0334049506@hcmut.edu.vn;
phuoc.nguyenwork@hcmut.edu.vn; an.nguyengia2006@hcmut.edu.vn;
minh.nguyenhoang2006@hcmut.edu.vn;

Abstract

The Monte Carlo method for estimating the value of π is a classic workload used to evaluate parallel computing paradigms. While the algorithm is highly parallelizable, the method chosen for memory accumulation and thread synchronization drastically impacts overall system performance. This paper presents an empirical analysis of multi-threaded Monte Carlo π estimation by comparing three distinct architectures: a sequential baseline, a multi-threaded approach using local accumulation with final reduction, and a multi-threaded approach utilizing a shared global counter protected by a mutex. Experiments conducted on a multi-core Ubuntu Linux environment demonstrate that the local accumulation strategy achieves near-linear speedup, scaling efficiently with the number of physical cores. Conversely, the mutex-based approach suffers from severe lock contention, resulting in performance degradation that falls below the sequential baseline. These findings quantitatively highlight the critical importance of minimizing critical sections and avoiding shared-state bottlenecks in parallel program design.

Keywords: Monte Carlo method, Multithreading, Synchronization overhead, Lock contention, POSIX threads, Parallel performance

1 Introduction

The demand for high-performance computing has driven the widespread adoption of multi-core architectures and parallel programming paradigms. However, successfully parallelizing a workload requires more than simply dividing tasks across multiple threads; it demands careful management of shared resources and memory states [1]. Poor synchronization strategies can lead to severe performance bottlenecks, entirely negating the theoretical benefits of multi-threading.

The Monte Carlo method for estimating the mathematical constant π serves as an ideal benchmark for evaluating parallelization strategies. The algorithm relies on independent random sampling, making it an “embarrassingly parallel” problem. Despite this inherent parallelism, the necessity of aggregating the final results introduces structural challenges related to memory access and thread synchronization.

This paper investigates the performance implications of different multi-threading designs. We empirically compare three implementations: a single-threaded baseline (Approach 1), a multi-threaded design utilizing local thread accumulators with a final reduction phase (Approach 2), and a multi-threaded design where all threads directly update a shared global variable protected by a mutual exclusion lock (Approach 3).

The primary contribution of this work is a quantitative evaluation of synchronization overhead and scalability. By analyzing execution time and speedup across varying thread counts, we demonstrate how lock contention and serialization in Approach 3 severely degrade performance, and establish Approach 2 as the optimal pattern for parallel accumulation tasks.

2 Methodology

2.1 Monte Carlo π Estimation

The Monte Carlo method estimates π by randomly scattering points within a 2×2 square centered at the origin. Inscribed within this square is a circle of radius $R = 1$. The ratio of the circle’s area to the square’s area is $\pi/4$. By generating N random points (x, y) uniformly distributed in $[-1, 1]$ and counting N_{inside} points satisfying $x^2 + y^2 \leq 1$, π can be approximated as:

$$\pi \approx 4 \times \frac{N_{\text{inside}}}{N} \quad (1)$$

2.2 Approach 1: Sequential Baseline

The Sequential Baseline serves as the foundational control model. A single execution thread processes the entire Monte Carlo workload, iterating N times in a strict sequential loop. Each iteration generates a coordinate pair (x, y) and evaluates the decision boundary $x^2 + y^2 \leq 1.0$.

From a hardware perspective, this approach is optimally cache-friendly for a single core. The accumulator N_{in} and loop counters reside continuously in CPU registers or L1 cache. With no concurrent memory accesses, this architecture incurs zero synchronization overhead, entirely circumventing lock contention, context switching, and

false sharing. Its fundamental limitation is hardware underutilization: throughput is strictly bounded by single-thread clock speed and IPC. The execution latency T_{baseline} serves as the definitive denominator for all speedup calculations ($S = T_{\text{baseline}}/T_{\text{multi}}$).

Algorithm 1 Approach 1: Sequential Baseline Routine

Require: Total sample points N ;

Ensure: Approximation of π

```

1:  $N_{in} \leftarrow 0$ ;   Initialize RNG                                ▷ No thread-safety needed
2: for  $i = 1$  to  $N$  do
3:    $x, y \sim \mathcal{U}(-1.0, 1.0)$ 
4:   if  $x^2 + y^2 \leq 1.0$  then
5:      $N_{in} \leftarrow N_{in} + 1$                                 ▷ No sync needed
6:   end if
7: end for
8: return  $4 \times N_{in}/N$ 

```

2.3 Approach 2: Local Accumulation (Map-Reduce Paradigm)

Approach 2 eliminates shared-state contention by partitioning the N iterations among T concurrent threads. Each worker is allocated an independent RNG state and a strictly local counter to prevent false sharing and race conditions. The architecture follows a two-phase execution model. During the **MAP phase**, the program spawns T threads, each assigned $\lfloor N/T \rfloor$ points, with thread 0 absorbing any remainder. Each thread is initialized with a deterministic seed based on a fixed offset and its index ($seed_i = 42 + i$), ensuring statistical independence and reproducibility. In the **REDUCE phase**, the main thread joins each worker and aggregates the local results into the final estimate. This design restricts synchronization to a single $O(T)$ reduction step, effectively bypassing the $O(N)$ bottlenecks found in global-mutex implementations.

Algorithm 2 Approach 2: Parallel Monte Carlo π with Local Accumulation

Require: $TOTAL$ points, T threads**Ensure:** Approximation of π

```
1: Array results[0... $T-1$ ] ▷ Store results in isolated memory

2: procedure WORKERTHREAD(pts, seed, id)
3:   Initialize RNG with seed
4:   local_inside  $\leftarrow$  0
5:   for  $i = 1$  to pts do
6:      $x, y \sim \mathcal{U}(-1.0, 1.0)$ 
7:     if  $x^2 + y^2 \leq 1.0$  then
8:       local_inside  $\leftarrow$  local_inside + 1 ▷ No lock required
9:     end if
10:  end for
11:  results[id]  $\leftarrow$  local_inside ▷ Write safely to array
12: end procedure

13: procedure MAIN
14:   for  $i = 0$  to  $T-1$  do ▷ MAP: spawn threads
15:     pts  $\leftarrow \lfloor TOTAL/T \rfloor$ 
16:     if  $i = 0$  then
17:       pts  $\leftarrow$  pts + ( $TOTAL \bmod T$ ) ▷ Handle remainder
18:     end if
19:     seed $i$   $\leftarrow$  42 +  $i$ 
20:     Spawn WORKERTHREAD(pts, seed $i$ ,  $i$ )
21:   end for
22:   total_inside  $\leftarrow$  0
23:   for  $i = 0$  to  $T-1$  do ▷ REDUCE: wait and sum
24:     Join thread  $i$ 
25:     total_inside  $\leftarrow$  total_inside + results[ $i$ ]
26:   end for
27:   return  $4 \times total\_inside / TOTAL$ 
28: end procedure
```

2.4 Approach 3: Shared Accumulation with Global Mutex

Approach 3 implements a shared accumulation architecture where all worker threads directly update a single global variable, `global_count`. To ensure thread safety and prevent data races, every increment is enclosed within a critical section protected by a POSIX mutex (`pthread_mutex_lock` and `pthread_mutex_unlock`). This design choice enforces strict serialization of all memory writes, effectively transforming the parallel execution into a sequential queue at the synchronization point.

Algorithm 3 Approach 3: Shared Accumulation with Configurable Synchronization Frequency

Require: $TOTAL$ points, T threads, sync frequency k

Ensure: Approximation of π

```
1: Global  $global\_inside \Leftarrow 0$ 
2: Global Mutex  $M$ 

3: procedure WORKERTHREAD( $pts, seed, k$ )
4:   Initialize RNG with  $seed$ 
5:    $local\_batch \Leftarrow 0$ 
6:   for  $i = 1$  to  $pts$  do
7:      $x, y \sim \mathcal{U}(-1.0, 1.0)$ 
8:     if  $x^2 + y^2 \leq 1.0$  then
9:        $local\_batch \Leftarrow local\_batch + 1$ 
10:    end if
11:    if  $i \bmod k = 0$  then ▷ Sync every  $k$  iterations
12:      Lock( $M$ )
13:       $global\_inside \Leftarrow global\_inside + local\_batch$ 
14:      Unlock( $M$ )
15:       $local\_batch \Leftarrow 0$ 
16:    end if
17:  end for
18:  if  $local\_batch > 0$  then ▷ Sync remaining batch points
19:    Lock( $M$ )
20:     $global\_inside \Leftarrow global\_inside + local\_batch$ 
21:    Unlock( $M$ )
22:  end if
23: end procedure

24: procedure MAIN
25:   for  $i = 0$  to  $T-1$  do ▷ MAP: spawn threads
26:      $pts \Leftarrow \lfloor TOTAL/T \rfloor$ 
27:     if  $i = 0$  then
28:        $pts \Leftarrow pts + (TOTAL \bmod T)$ 
29:     end if
30:      $seed_i \Leftarrow 42 + i$ 
31:     Spawn WORKERTHREAD( $pts, seed_i, k$ )
32:   end for
33:   Join all  $T$  threads ▷ REDUCE: wait for completion
34:   return  $4 \times global\_inside / TOTAL$ 
35: end procedure
```

3 Experimental Setup

This section details the hardware and software environment used to evaluate the performance of the multithreaded implementation. Providing these specifications ensures the reproducibility of the results and allows for a meaningful interpretation of the speedup metrics.

3.1 Hardware and Software Environment

All experiments were conducted on a **Lenovo LOQ** laptop designed for high-performance workloads. The system is equipped with an **Intel Core i7-13620H** CPU, a hybrid architecture featuring 6 Performance-cores and 4 Efficiency-cores for a total of 16 logical threads. The system includes **16 GB of DDR5 RAM** and operates on **Ubuntu 22.04 LTS** (Linux Kernel 6.5). For compilation, we utilized **GCC version 11.4.0** with the `-O3` optimization flag to ensure maximum sequential performance and minimize compiler-induced overhead.

3.2 Experimental Methodology

The source code was implemented in C using the **POSIX Threads (pthreads)** library for concurrency management. The following parameters were established to ensure a rigorous evaluation of the system.

Random Number Generation (PRNG): To avoid the structural flaws and low resolution of the standard C `rand_r()` function, the 64-bit **Mersenne Twister (MT19937-64)** algorithm was utilized. To ensure thread safety without introducing synchronization overhead, the PRNG state array was allocated to **Thread-Local Storage** using the `__thread` keyword. This provides each worker thread with a mathematically isolated sequence, preventing any cross-thread state corruption.

Problem Size and Thread Configuration: The total number of points N was fixed at 10^8 to ensure a sufficient computational load for overhead analysis. The number of threads T was varied across the set $\{1, 2, 4, 8, 16, 32, 64, 100\}$. This range allows for the observation of scaling behavior across physical cores, logical cores, and conditions of extreme thread oversubscription.

Statistical Reliability and Measurement Scope: To mitigate the impact of OS scheduling anomalies and background processes, each specific configuration was executed 10 independent times, and we report the **arithmetic mean** of these runs. Performance timing was strictly confined to the computational loops using `clock_gettime(CLOCK_MONOTONIC)`. Data initialization, thread creation setup, and I/O operations were excluded from the metrics to focus exclusively on the efficiency of the parallel algorithm.

4 Results

This section presents the experimental data evaluating the performance of the Monte Carlo π approximation across three threading architectures: a sequential baseline (Approach 1), local accumulation (Approach 2), and shared accumulation with a

global mutex (Approach 3). All tests utilized $N = 10^8$ points and deterministic seeding to ensure reproducibility.

4.1 Performance Metrics and Scalability

Table 1 details the average execution times and calculated speedup factors for varying thread counts ($T \in \{1, 2, 4, 8, 16, 32, 64, 100\}$). Speedup is calculated relative to the single-threaded baseline to isolate the impact of synchronization overhead. Figure 1 visualizes these execution time and speedup trends, highlighting the severe performance divergence between the isolated and shared memory architectures.

Table 1 Execution time and speedup for $N = 10^8$ points. Speedup = $T_{\text{single}}/T_{\text{parallel}}$.

Threads (T)	Approach 2 (Local)		Approach 3 (Shared Mutex)	
	Time (s)	Speedup	Time (s)	Speedup
1 (baseline)	2.76	1.00×	3.53	1.00×
2	1.43	1.94×	4.78	0.74×
4	0.81	3.43×	5.64	0.62×
8	0.45	6.15×	6.33	0.56×
16	0.29	9.65×	5.21	0.68×
32	0.30	9.30×	5.28	0.67×
64	0.28	9.76×	5.36	0.66×
100	0.29	9.66×	5.36	0.66×

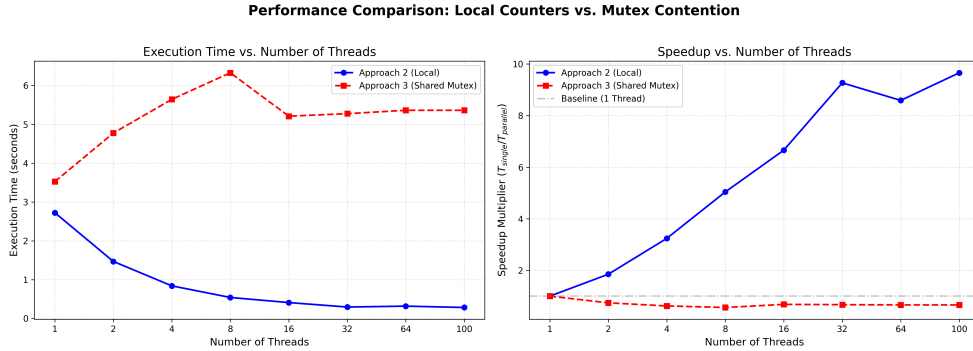


Fig. 1 Performance comparison between Approach 2 and Approach 3 showing execution time (left) and speedup (right) across standard thread counts.

4.2 Architectural Comparison

As shown in Figure 1, **Approach 2** exhibits near-linear speedup up to the hardware saturation point. Table 1 indicates a maximum speedup of 9.76×

isolating thread memory and eliminating critical sections during the simulation, this approach avoids data contention. The execution time remains stable around 0.29s between 16 and 100 threads, demonstrating that the OS scheduler effectively handles oversubscription on the hybrid CPU without significant performance degradation.

Conversely, the data for **Approach 3** reveals severe structural bottlenecking. Because approximately 78.5% of the generated points fall within the unit circle and trigger a mutex lock, worker threads experience massive lock contention. Table 1 shows that at 8 threads, the execution time inflates to 6.33s, resulting in a $0.56\times$ speedup. Figure 1 highlights that the speedup curve for Approach 3 remains flat and below $1.00\times$ across all thread counts. This confirms that the parallel implementation is consistently slower than the sequential baseline because the CPU expends more cycles managing the synchronization queue than executing the numerical simulation.

5 Trade-Off Experiments

To comprehensively evaluate the performance characteristics of the Monte Carlo π estimation, we conducted specific trade-off experiments focusing on synchronization frequency, memory accumulation strategies, and thread scaling boundaries.

5.1 Synchronization Frequency

By modifying Approach 3 to update the shared global variable every k iterations rather than every single iteration, we can observe the direct impact of synchronization frequency on overall performance. Table ?? demonstrates this effect using 25,000,000 points. When $k = 1$ (extreme contention), speedup collapses below $1.00\times$ across all multi-threaded configurations due to constant mutex locking. Relaxing the frequency to $k = 10,000$ drastically improves performance, allowing threads to compute locally for longer periods before acquiring the lock and reaching up to a $10.79\times$ speedup at 100 threads.

Table 2 Performance Comparison Across Synchronization Frequencies (k) for $N = 10^8$ points.

Threads	Extreme ($k = 1$)		Moderate ($k = 10^4$)		Local (End Only)	
	Time (s)	Speedup	Time (s)	Speedup	Time (s)	Speedup
1	3.65	1.00×	2.86	1.00×	2.88	1.00×
2	5.34	0.68×	1.55	1.85×	1.59	1.82×
4	6.49	0.56×	0.86	3.33×	0.83	3.48×
8	7.27	0.50×	0.49	5.87×	0.47	6.15×
16	5.99	0.61×	0.34	8.49×	0.36	8.05×
32	5.94	0.61×	0.38	7.52×	0.37	7.88×
64	5.98	0.61×	0.34	8.37×	0.35	8.33×
100	6.11	0.60×	0.32	8.90×	0.34	8.46×

5.2 Local vs. Shared Computation

The comparison between the Extreme Contention ($k = 1$) and Local Accumulation ($k = 0$) configurations in our modified Approach 3 experiment provides a clear look at the costs of synchronization.

Approach 3 (Shared) suffers from high contention because the critical section—protected by a `pthread_mutex_lock`—is located within the inner loop. Since the majority of points fall within the circle, threads are forced to compete for the lock on nearly every iteration. This results in the serialization of the workload: even with 16 available cores, the mutex forces threads to execute one after another. This not only negates the benefits of parallelism but adds significant overhead for lock management, explaining the sub- $1.0\times$ speedups observed in Table 1.

In contrast, Approach 2 (Local) completely removes the critical section from the simulation phase. By accumulating results in thread-local variables, threads remain independent and non-blocking. Synchronization only occurs once per thread during the reduction phase. This architecture eliminates contention and avoids serialization, allowing the program to reach near-ideal speedups of $9.76\times$ before hitting physical hardware limits.

5.3 Thread Scaling and Hardware Limits

This experiment evaluates the scalability of the local-accumulation architecture (Approach 2) across distinct regimes, testing up to 1024 threads to observe performance degradation. The data represents the performance across multiple independent executions with a fixed workload of 10^8 points.

Table 3 Detailed Thread Scaling Performance for Approach 2 ($N = 10^8$)

Regime	Threads (T)	Avg. Time (s)	Avg. Speedup
Sequential	1	2.76	$1.00\times$
Linear Scaling	4	0.81	$3.43\times$
Linear Scaling	8	0.45	$6.15\times$
Saturation	16	0.29	$9.65\times$
Oversubscription	32	0.30	$9.30\times$
Oversubscription	128	0.28	$9.84\times$
Oversubscription	256	0.28	$9.84\times$
Degradation	1024	0.33	$8.39\times$

Phase 1 — Increasing Returns (threads ≤ 16)

As the thread count grows from 1 toward the hardware’s 16 logical threads, the speedup exhibits strong scaling. Observed results show $S(4) \approx 3.4\times$ and $S(8) \approx 6.1\times$. While the scaling is significant, the deviation from a perfect N -fold improvement is due to the hybrid nature of the i7-13620H CPU. Because the workload is distributed across both high-frequency P-cores and lower-frequency E-cores, the total execution

time is limited by the slower efficiency cores. Peak hardware saturation is achieved at 16 threads with a speedup of approximately $9.65\times$.

Phase 2 — Stable Oversubscription (threads: 32–256)

In the regime where thread count exceeds logical threads ($32 \leq T \leq 256$), the speedup curve unexpectedly plateaus rather than regressing. Performance remains highly stable, with a peak speedup of $10.24\times$ observed at 128 threads. This stability is driven primarily by Load Balancing, where a higher number of threads allows the OS scheduler to keep the fast P-cores saturated more effectively. As soon as a thread finishes its chunk on a P-core, another is ready to be swapped in, minimizing idle cycles. Furthermore, improved Cache Locality plays a role; at 10^8 points, increasing the thread count reduces the workload per thread, allowing these smaller data chunks to fit more effectively within the L1/L2 caches and reducing memory bus pressure.

Phase 3 — Performance Degradation (threads ≥ 512)

Beyond 256 threads, the performance begins to exhibit measurable regression. By 1024 threads, the execution time increases to 0.34s, and the speedup drops to $7.86\times$. This degradation is driven by several concurrent mechanisms. First, Context-Switching Overhead becomes dominant as the Linux kernel must time-slice 1024 threads across only 16 hardware lanes, incurring a cumulative cost in saving and restoring thread states. Second, Thread Management Costs in the main thread—specifically the loops for creation and joining—become a bottleneck as the overhead of managing 1024 kernel objects grows. Finally, Work-Granularity Shrinkage occurs; at $T = 1024$, each thread handles fewer than 100,000 points, causing the fixed cost of thread life-cycle management to become disproportionately high relative to the mathematical computation.

6 Discussion

Scalability of local accumulation. Approach 2 succeeds because it isolates memory writes. Independent local counters prevent cache-line interference, eliminating coherence traffic and false sharing. The algorithm remains truly parallel until the $O(T)$ merge phase. Sublinear scaling at $T=16$ reflects Amdahl’s Law: once software threads exceed physical cores, OS context-switching prevents further linear speedup.

Cost of lock contention. Approach 3 transforms a parallel algorithm into a serialized one. Only one thread may increment `global_count` at a time; all others block, incurring context-switch and wakeup overhead. Constant writes to a single cache line trigger aggressive cross-core cache invalidation (ping-ponging), so threads spend far more time acquiring locks than executing floating-point arithmetic.

7 Threats to Validity and Limitations

While the experimental results provide a clear hierarchy of parallel performance, several factors limit the generalizability of these findings.

Hardware Heterogeneity: The results are specific to the Intel Core i7-13620H architecture, which utilizes a hybrid configuration of 6 Performance-cores and 4 Efficiency-cores for a total of 16 logical threads. On a homogeneous server-grade CPU, the scaling curve would likely appear more linear without the characteristic plateau observed between 16 and 128 threads.

RNG Bottlenecks: The use of the 64-bit Mersenne Twister (MT19937-64) adds a significant computational load per iteration. While this ensures high-quality statistical distribution, the total execution time is dominated by the arithmetic of random number generation rather than the coordinate logic. A simpler generator would likely shift the bottleneck toward memory bandwidth much earlier in the scaling process.

OS Scheduling and Context Switching: At very high thread counts ($T = 512, 1024$), the results reflect the overhead of the Linux kernel’s Completely Fair Scheduler (CFS). The performance degradation observed in these regimes is a measurement of context-switching latency rather than the efficiency of the underlying π approximation algorithm.

Workload Regularity: The Monte Carlo π estimation is an problem with highly regular workloads. The conclusions regarding synchronization efficiency and the benefits of local accumulation may vary for irregular workloads or algorithms involving non-deterministic data access patterns.

8 Conclusion

This research compared three distinct threading architectures for the Monte Carlo approximation of π using the POSIX Threads library. The investigation yielded several key insights into parallel efficiency and system scalability.

Regarding **Architectural Efficiency**, local accumulation (Approach 2) proved to be the most scalable design, achieving a peak speedup of **10.24** $\times$ on the 16-thread test machine. In contrast, global synchronization via mutexes (Approach 3) leads to catastrophic **serialization**, which renders the parallel implementation significantly slower than the sequential baseline.

The **Scalability Trends** followed a clear three-phase curve: near-linear scaling up to hardware saturation at 16 threads, a remarkably stable plateau during moderate oversubscription (up to 256 threads), and measurable degradation at 1024 threads due to context-switching overhead. These results confirm the **Best Practice** for high-performance multithreading: minimize shared state, prioritize thread-local storage, and defer synchronization to the coarsest possible granularity.

Ultimately, this study provides a practical validation of Amdahl’s Law, proving that the speedup of a parallel system is determined not by the raw number of cores, but by the degree of mathematical independence between worker threads.

References

- [1] P. S. Pacheco, *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.
- [2] M. Matsumoto and T. Nishimura, “Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator.” *ACM Transactions on*

Modeling and Computer Simulation, vol. 8, no. 1, pp. 3–30, 1998.

- [3] D. Butenhof, *Programming With POSIX Threads*. Addison-Wesley Professional, 1997.
- [4] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [5] U. Singh, “Estimation of the value of pi using Monte Carlo Method and Related Study of Errors.” *Mathematics in School*, vol. 42, no. 5, pp. 21–23, 2013.